

Theory of Computation

Languages and Finite Automata

Samit Biswas¹

¹Department of Computer Science and Technology,
Indian Institute of Engineering Science and Technology, Shibpur
Email: samit@cs.iiests.ac.in

Table of Contents

- 1 Introduction to Strings
- 2 Languages
- 3 Finite Automata
- 4 Languages Accepted by DFAs

Language

- A language is a set of **strings**.

Language

- A language is a set of **strings**.
- **Strings:** A sequence of letters
Example: “cat”, “dog”, “house”, ...

Language

- A language is a set of **strings**.
- **Strings:** A sequence of letters
Example: “cat”, “dog”, “house”, ...
- Defined over an **alphabet**, Σ :
 $\Sigma = \{a, b, c, d, \dots, z\}$

- A language is a set of **strings**.
- **Strings:** A sequence of letters
Example: “cat”, “dog”, “house”, ...
- Defined over an **alphabet**, Σ :
 $\Sigma = \{a, b, c, d, \dots, z\}$
- More Examples
 - **0011** and **11** are strings over $\Sigma = \{0, 1\}$
 - **abc** and **bbb** are strings over $\Sigma = \{a, b, \dots, z\}$
 - **((()()))** and **)(()** are strings over $\Sigma = \{ (,) \}$

Alphabets and Strings

- We will use small alphabets: $\Sigma = \{a, b\}$

Alphabets and Strings

- We will use small alphabets: $\Sigma = \{a, b\}$

- **Strings:**

a	
ab	$u = ab$
abba	$v = bbbaaa$
baba	$w = abba$
aaabbbbaabab	

String Operations

- Consider two strings: w & v over $\Sigma = \{a, b\}$

$w = a_1 a_2 a_3 \dots a_n$		abba
$v = b_1 b_2 b_3 \dots b_n$		bbbbaaa

String Operations

- Consider two strings: w & v over $\Sigma = \{a, b\}$

$w = a_1 a_2 a_3 \dots a_n$ | abba

$v = b_1 b_2 b_3 \dots b_n$ | bbbaaa

- Concatenation:**

$wv = a_1 a_2 a_3 \dots a_n b_1 b_2 b_3 \dots b_n$ | abbabbbaaa

String Operations

- Consider two strings: w & v over $\Sigma = \{a, b\}$

$$w = a_1 a_2 a_3 \dots a_n \quad | \quad \text{abba}$$

$$v = b_1 b_2 b_3 \dots b_n \quad | \quad \text{bbbaaa}$$

- Concatenation:**

$$wv = a_1 a_2 a_3 \dots a_n b_1 b_2 b_3 \dots b_n \quad | \quad \text{abbabbbbaaa}$$

- Reverse:**

Consider the following string:

$$w = a_1 a_2 a_3 \dots a_n \quad | \quad \text{ababaaabbb}$$

After reverse:

$$w^R = a_n \dots a_3 a_2 a_1 \quad | \quad \text{bbbaaababa}$$

String Operations

- Consider two strings: w & v over $\Sigma = \{a, b\}$

$$\begin{array}{l|l} w = a_1 a_2 a_3 \dots a_n & \text{abba} \\ v = b_1 b_2 b_3 \dots b_n & \text{bbbaaa} \end{array}$$

- Concatenation:**

$$wv = a_1 a_2 a_3 \dots a_n b_1 b_2 b_3 \dots b_n \quad | \quad \text{abbabbbbaaa}$$

- Reverse:**

Consider the following string:

$$w = a_1 a_2 a_3 \dots a_n \quad | \quad \text{ababaaabbb}$$

After reverse:

$$w^R = a_n \dots a_3 a_2 a_1 \quad | \quad \text{bbbaaababa}$$

- String Length:** $w = a_1 a_2 a_3 \dots a_n$

$$\text{Length: } |w| = n$$

Examples:

$$|abba| = 4$$

$$|aa| = 2$$

$$|a| = 1$$

String Operations

- **Recursive Definition of Length**

For any letter

$$|a| = 1$$

For any String

$$|wa| = |w| + 1$$

String Operations

- Recursive Definition of Length

For any letter

$$|a| = 1$$

For any String

$$|wa| = |w| + 1$$

Example: $|abba|$

$$= |abb| + 1$$

$$= |ab| + 1 + 1$$

$$= |a| + 1 + 1 + 1$$

$$= 1 + 1 + 1 + 1$$

$$= 4$$

String Operations

- Recursive Definition of Length**

For any letter	$ a = 1$
For any String	$ wa = w + 1$

Example: $ abba $	$= abb + 1$
	$= ab + 1 + 1$
	$= a + 1 + 1 + 1$
	$= 1 + 1 + 1 + 1$
	$= 4$

- Length of Concatenation:** $|uv| = |u| + |v|$

$u = aab$	$ u = 3$
$v = abaab$	$ v = 5$
$ uv = aababaab $	$= 8$
	$ uv = u + v = 3 + 5 = 8$

- **Proof of Concatenation Length**

Claim: $|uv| = |u| + |v|$

Proof: By induction on the length $|v|$

- **Proof of Concatenation Length**

Claim: $|uv| = |u| + |v|$

Proof: By induction on the length $|v|$

Induction basis: $|v| = 1$

From the definition of length: $|uv| = |u| + 1 = |u| + |v|$

- **Proof of Concatenation Length**

Claim: $|uv| = |u| + |v|$

Proof: By induction on the length $|v|$

Induction basis: $|v| = 1$

From the definition of length: $|uv| = |u| + 1 = |u| + |v|$

Inductive hypothesis: $|uv| = |u| + |v|$ for $|v| = 1, 2, 3, \dots, n$

- **Proof of Concatenation Length**

Claim: $|uv| = |u| + |v|$

Proof: By induction on the length $|v|$

Induction basis: $|v| = 1$

From the definition of length: $|uv| = |u| + 1 = |u| + |v|$

Inductive hypothesis: $|uv| = |u| + |v|$ for $|v| = 1, 2, 3, \dots, n$

Inductive step: we will prove $|uv| = |u| + |v|$ for $|v| = n + 1$

Write $v = wa$, where $|w| = n$, $|a| = 1$

from definition of length $|uv| = |uwa| = |uw| + 1$; $|wa| = |w| + 1$

From inductive hypothesis: $|uw| = |u| + |w|$

Thus, $|uv| = |u| + |w| + 1 = |u| + |wa| = |u| + |v|$

String Operations

- **Empty String:** A string with no letters, λ

Observations: $|\lambda| = 0$

$$\lambda w = w\lambda = w$$

$$\lambda abba = abba\lambda = abba$$

String Operations

- **Empty String:** A string with no letters, λ

Observations: $|\lambda| = 0$

$$\lambda w = w\lambda = w$$

$$\lambda abba = abba\lambda = abba$$

- **Substring of string:** a subsequence of consecutive characters

String	Substring
<u>a</u> bbab	ab
ab <u>b</u> ab	abba
abb <u>a</u> b	b
abba <u>b</u>	bbab

String Operations

- **Empty String:** A string with no letters, λ

Observations: $|\lambda| = 0$

$$\lambda w = w\lambda = w$$

$$\lambda abba = abba\lambda = abba$$

- **Substring of string:** a subsequence of consecutive characters

String	Substring
<u>a</u> bbab	ab
ab <u>b</u> ab	abba
abb <u>a</u> b	b
abba <u>b</u>	bbab

- **Prefix and Suffix:** Consider a String: *abbab*

Prefixes	Suffixes
λ	<i>abbab</i>
a	bbab
ab	bab
abb	ab
abba	b

String Operations

- **Another Operation:**

$$w^n = \underbrace{www \dots w}_n$$

Example: $(abba)^2 = abbaabba$

Definition: $w^0 = \lambda$

$$(abba)^0 = \lambda$$

String Operations

- **Another Operation:**

$$w^n = \underbrace{www \dots w}_n$$

Example: $(abba)^2 = abbaabba$

Definition: $w^0 = \lambda$

$$(abba)^0 = \lambda$$

- **The * Operation:**

Σ^* = the set of all possible strings from alphabet Σ

$$\Sigma = \{a, b\}$$

$$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

String Operations

- **Another Operation:**

$$w^n = \underbrace{www \dots w}_n$$

Example: $(abba)^2 = abbaabba$

Definition: $w^0 = \lambda$

$$(abba)^0 = \lambda$$

- **The * Operation:**

Σ^* = the set of all possible strings from alphabet Σ

$$\Sigma = \{a, b\}$$

$$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

- **The + Operation:**

Σ^+ = the set of all possible strings from alphabet Σ except λ

$$\Sigma = \{a, b\}$$

$$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

$$\Sigma^+ = \Sigma^* - \lambda$$

$$\Sigma^+ = \{a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

Table of Contents

- 1 Introduction to Strings
- 2 Languages
- 3 Finite Automata
- 4 Languages Accepted by DFAs

Language

- A **language** is any subset of over Σ^* .

- **Example:**

$$\Sigma = \{a, b\}$$

$$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

- **Languages:**

$$\{\lambda\}$$

$$\{a, aa, aab\}$$

$$\{\lambda, a, abba, baba, aa, ab, aaaaaa\}$$

Language

- A **language** is any subset of over Σ^* .
 - **Example:**
 $\Sigma = \{a, b\}$
 $\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$
 - **Languages:**
 $\{\lambda\}$
 $\{a, aa, aab\}$
 $\{\lambda, a, abba, baba, aa, ab, aaaaaa\}$
 - An **infinite language** $L = \{a^n b^n : n \geq 0\}$

$$\left. \begin{array}{l} \lambda \\ ab \\ aabb \\ aaaaabbbbb \end{array} \right\} \in L$$

$$abb \notin L$$

Operations on Languages

- **The usual set operations:**

$$\{a, ab, aaaa\} \cup \{bb, ab\} = \{a, ab, bb, aaaa\}$$

$$\{a, ab, aaaa\} \cap \{bb, ab\} = \{ab\}$$

$$\{a, ab, aaaa\} - \{bb, ab\} = \{a, aaaa\}$$

Operations on Languages

- **The usual set operations:**

$$\{a, ab, aaaa\} \cup \{bb, ab\} = \{a, ab, bb, aaaa\}$$

$$\{a, ab, aaaa\} \cap \{bb, ab\} = \{ab\}$$

$$\{a, ab, aaaa\} - \{bb, ab\} = \{a, aaaa\}$$

- **Complement:** $\overline{L} = \Sigma^* - L$

$$\overline{\{a, ba\}} = \{\lambda, b, aa, ab, bb, aaa, \dots\}$$

Operations on Languages

- **The usual set operations:**

$$\{a, ab, aaaa\} \cup \{bb, ab\} = \{a, ab, bb, aaaa\}$$

$$\{a, ab, aaaa\} \cap \{bb, ab\} = \{ab\}$$

$$\{a, ab, aaaa\} - \{bb, ab\} = \{a, aaaa\}$$

- **Complement:** $\overline{L} = \Sigma^* - L$

$$\overline{\{a, ba\}} = \{\lambda, b, aa, ab, bb, aaa, \dots\}$$

- **Reverse:** $L^R = \{w^R : w \in L\}$

$$\{ab, aab, baba\}^R = \{ba, baa, abab\}$$

$$L = \{a^n b^n : n \geq 0\}$$

$$L^R = \{b^n a^n : n \geq 0\}$$

Operations on Languages

- **The usual set operations:**

$$\{a, ab, aaaa\} \cup \{bb, ab\} = \{a, ab, bb, aaaa\}$$

$$\{a, ab, aaaa\} \cap \{bb, ab\} = \{ab\}$$

$$\{a, ab, aaaa\} - \{bb, ab\} = \{a, aaaa\}$$

- **Complement:** $\bar{L} = \Sigma^* - L$

$$\overline{\{a, ba\}} = \{\lambda, b, aa, ab, bb, aaa, \dots\}$$

- **Reverse:** $L^R = \{w^R : w \in L\}$

$$\{ab, aab, baba\}^R = \{ba, baa, abab\}$$

$$L = \{a^n b^n : n \geq 0\}$$

$$L^R = \{b^n a^n : n \geq 0\}$$

- **Concatenation:** $L_1 L_2 = \{xy : x \in L_1, y \in L_2\}$

$$\{a, ab, ba\} \{b, aa\} = \{ab, aaa, abb, abaa, bab, baaa\}$$

Operations on Languages

- **The usual set operations:**

$$\{a, ab, aaaa\} \cup \{bb, ab\} = \{a, ab, bb, aaaa\}$$

$$\{a, ab, aaaa\} \cap \{bb, ab\} = \{ab\}$$

$$\{a, ab, aaaa\} - \{bb, ab\} = \{a, aaaa\}$$

- **Complement:** $\overline{L} = \Sigma^* - L$

$$\overline{\{a, ba\}} = \{\lambda, b, aa, ab, bb, aaa, \dots\}$$

- **Reverse:** $L^R = \{w^R : w \in L\}$

$$\{ab, aab, baba\}^R = \{ba, baa, abab\}$$

$$L = \{a^n b^n : n \geq 0\}$$

$$L^R = \{b^n a^n : n \geq 0\}$$

- **Concatenation:** $L_1 L_2 = \{xy : x \in L_1, y \in L_2\}$

$$\{a, ab, ba\} \{b, aa\} = \{ab, aaa, abb, abaa, bab, baaa\}$$

- **Another Operation:** $L^n = \underbrace{LL \dots L}_n$

$$\{a, b\}^3 = \{a, b\} \{a, b\} \{a, b\} =$$

$$\{aaa, aab, aba, abb, baa, bab, bba, bbb\}$$

$$L^0 = \{\lambda\} ; \{a, bb, aa\}^0 = \{\lambda\}$$

Operations on Languages

- **Star-Closure (Kleene*) :** $L^* = L^0 \cup L^1 \cup L^2 \dots$

Example:

$$\{a, bb\}^* = \left\{ \begin{array}{l} \lambda, \\ a, bb, \\ aa, abb, bba, bbbb, \\ aaa, aabb, abba, abbbb, \dots \end{array} \right\}$$

- **Positive Closure:** $L^+ = L^1 \cup L^2 \cup L^3 \dots = L^* - \{\lambda\}$

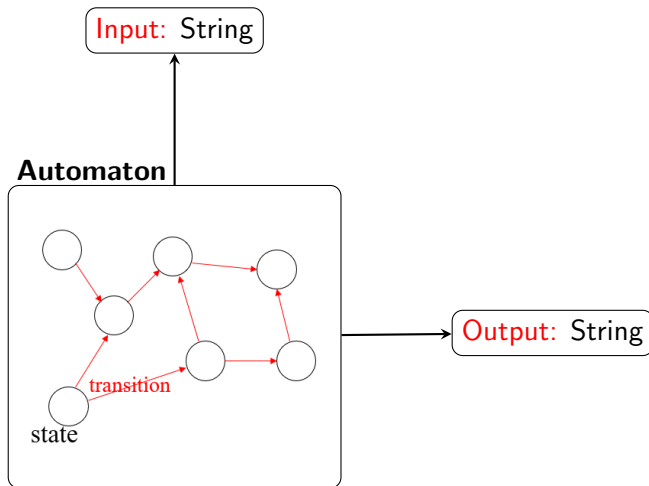
Example:

$$\{a, bb\}^+ = \left\{ \begin{array}{l} a, bb, \\ aa, abb, bba, bbbb, \\ aaa, aabb, abba, abbbb, \dots \end{array} \right\}$$

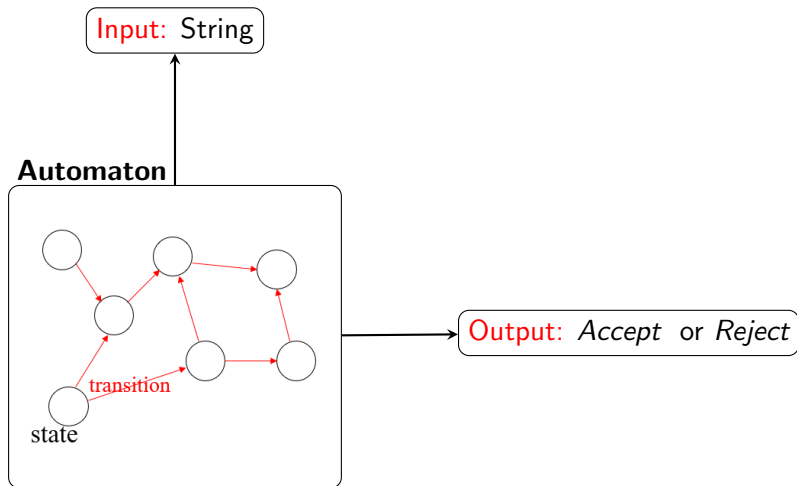
Table of Contents

- 1 Introduction to Strings
- 2 Languages
- 3 Finite Automata**
- 4 Languages Accepted by DFAs

Finite Automata



Finite Acceptor



Finite Automata or Finite State Machines

- Problem is to decide whether a given string is a member of some particular language or not.

Finite Automata or Finite State Machines

- Problem is to decide whether a given string is a member of some particular language or not.

Types of Finite State Machines

- Finite Automata (Accept / Reject):
 - Deterministic Finite Automata (DFA)
 - Non-deterministic Finite Automata (NFA)
 - Finite Automata with ϵ -transitions (ϵ -NFA)
- Finite Automata (Generate an output sequence):
 - Mealy And Moore Machine

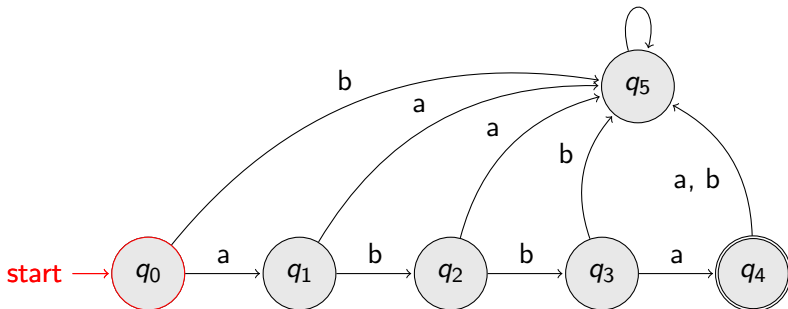
Transition Graph

- **abba** -Finite Acceptor:

↓
Input String:

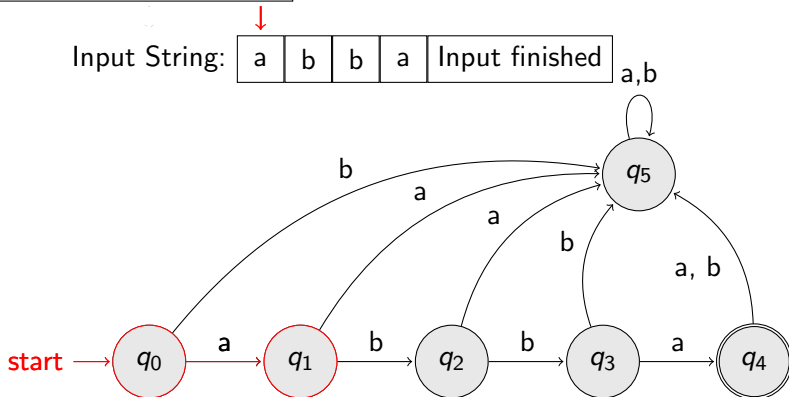
a	b	b	a
---	---	---	---

 Input finished a,b



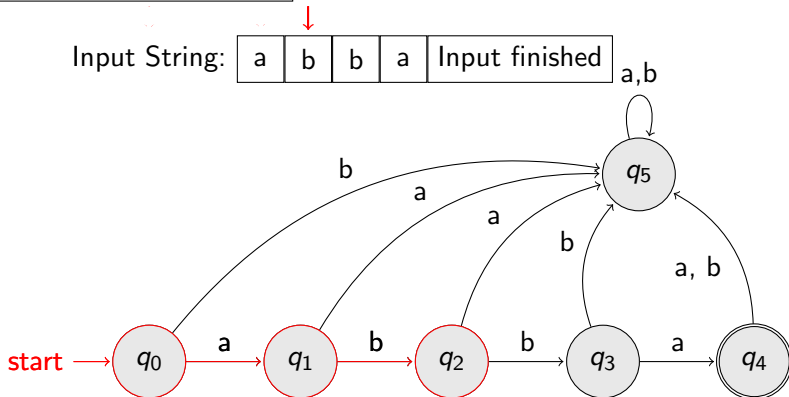
Transition Graph

- **abba** -Finite Acceptor:



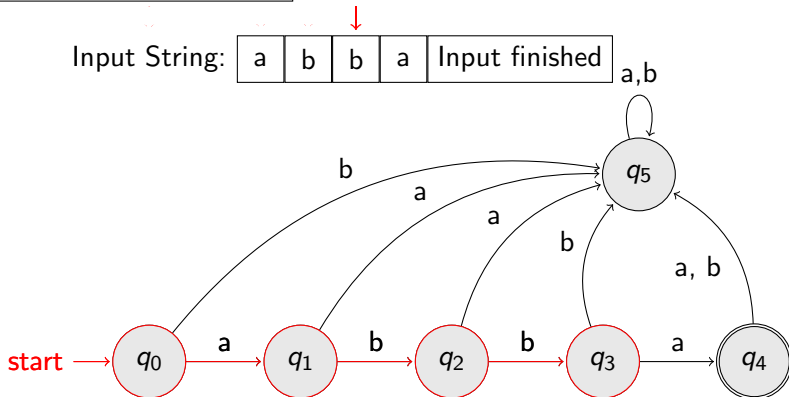
Transition Graph

- **abba** -Finite Acceptor:



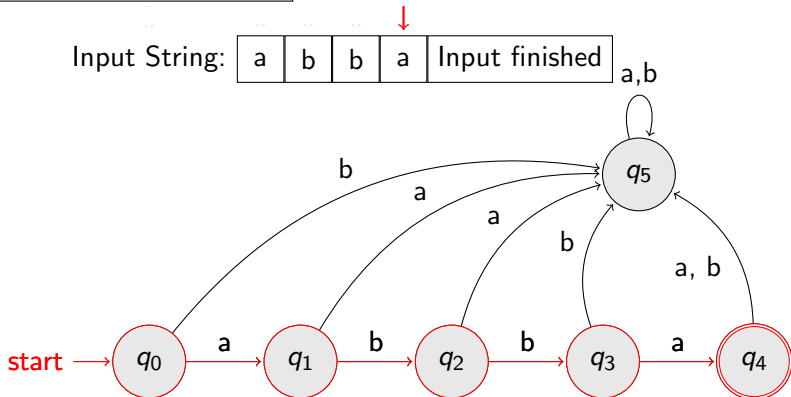
Transition Graph

- **abba** -Finite Acceptor:



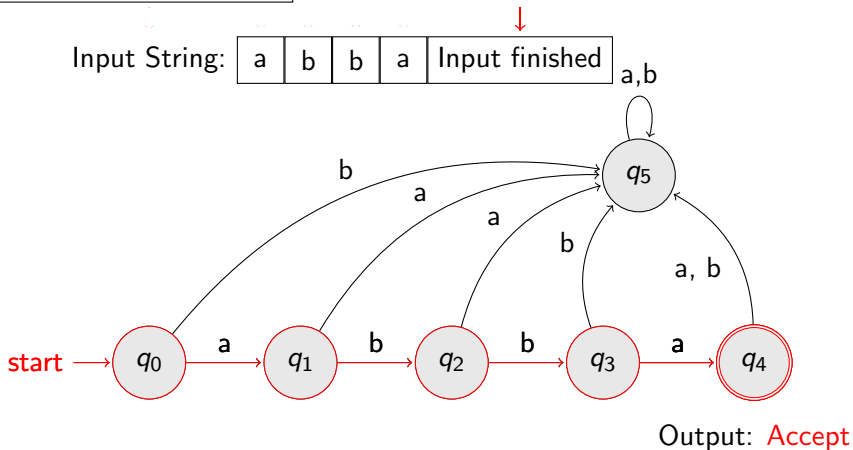
Transition Graph

- **abba** -Finite Acceptor:



Transition Graph

- **abba** -Finite Acceptor:

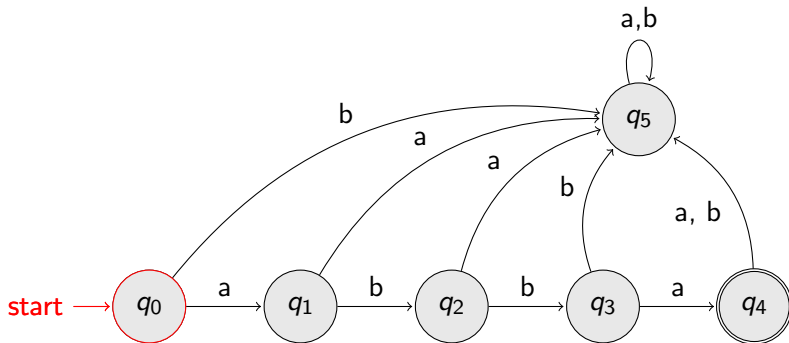


Transition Graph

- **aba** - Reject:

↓
Input String:

a	b	a	Input finished
---	---	---	----------------



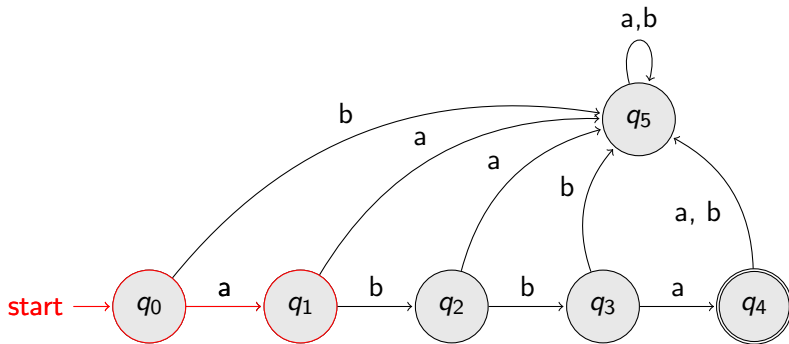
Transition Graph

- **aba** - Reject:

Input String:

a	b	a
---	---	---

 Input finished



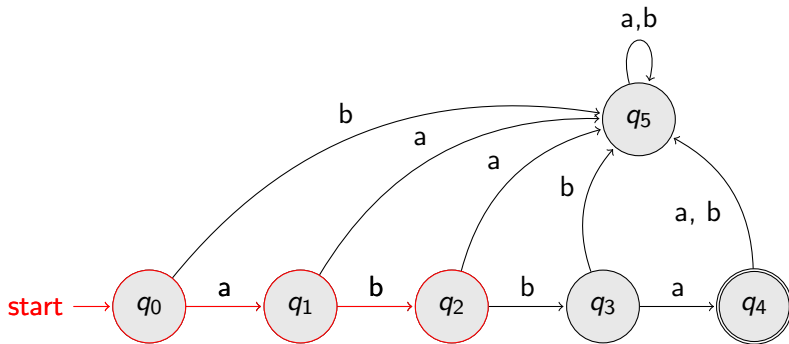
Transition Graph

- **aba** - Reject:

Input String:

a	b	a
---	---	---

 Input finished



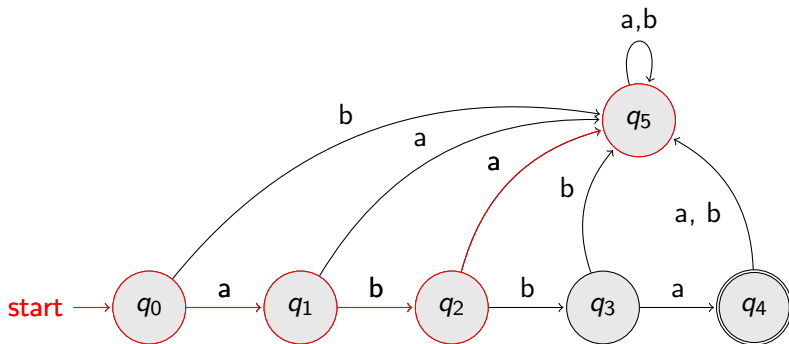
Transition Graph

- **aba** - Reject:

Input String:

a	b	a
---	---	---

 Input finished

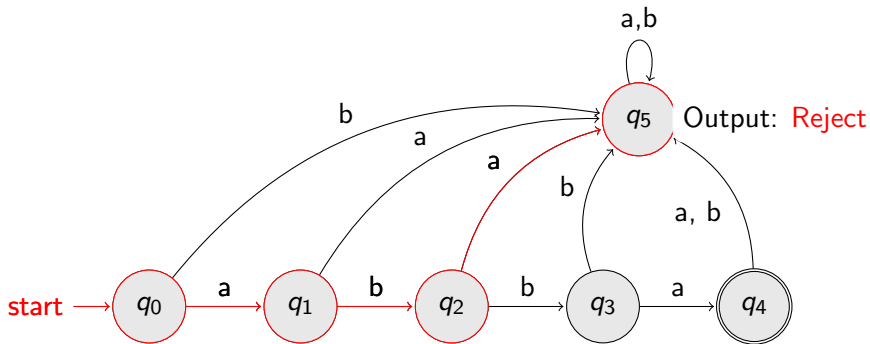


Transition Graph

- **aba** - Reject:

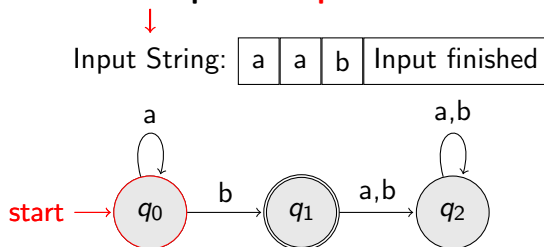
Input String:

a	b	a	Input finished
---	---	---	----------------



Transition Graph

- Another Example: **Accept**

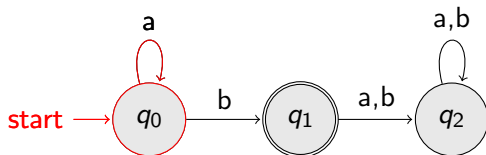


Transition Graph

- **Another Example: Accept**

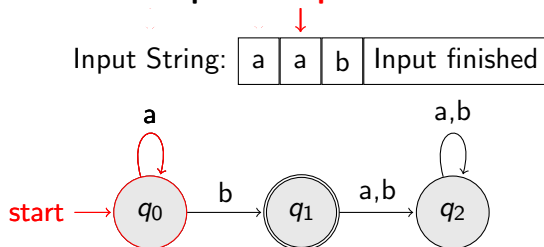
Input String:

a	a	b	Input finished
---	---	---	----------------



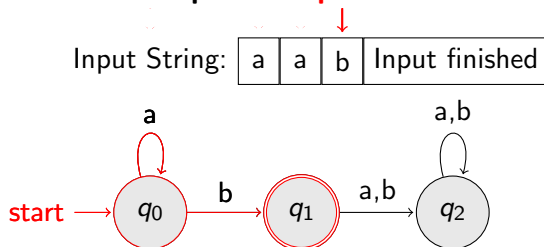
Transition Graph

- **Another Example: Accept**



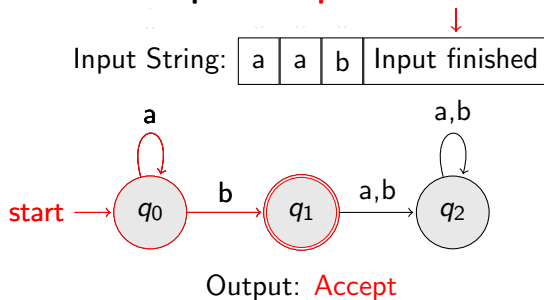
Transition Graph

- Another Example: **Accept**



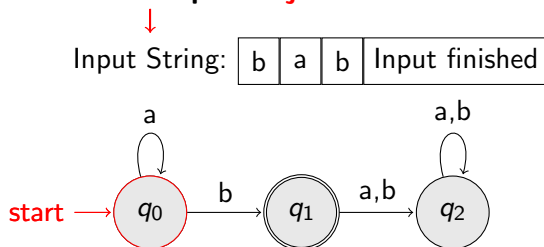
Transition Graph

- Another Example: **Accept**



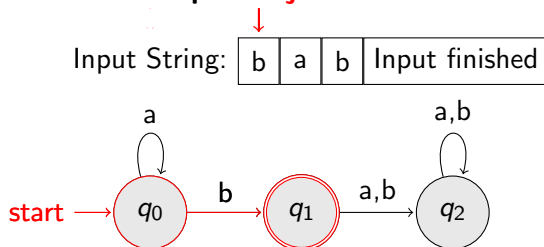
Transition Graph

- Another Example: **Rejection**



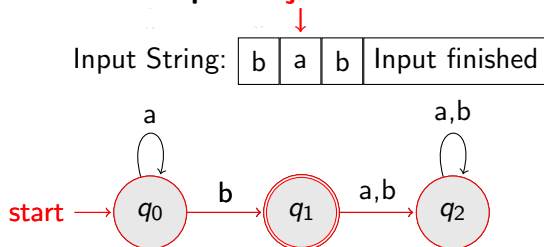
Transition Graph

- Another Example: **Rejection**



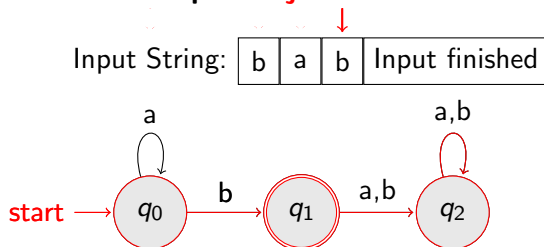
Transition Graph

- Another Example: **Rejection**



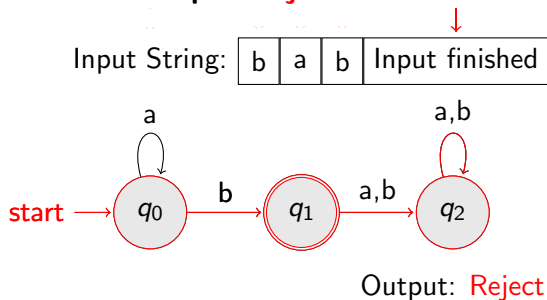
Transition Graph

- Another Example: **Rejection**



Transition Graph

- Another Example: **Rejection**



- **Deterministic Finite Automata (DFA):**

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q : Finite set of states

Σ : Input alphabet

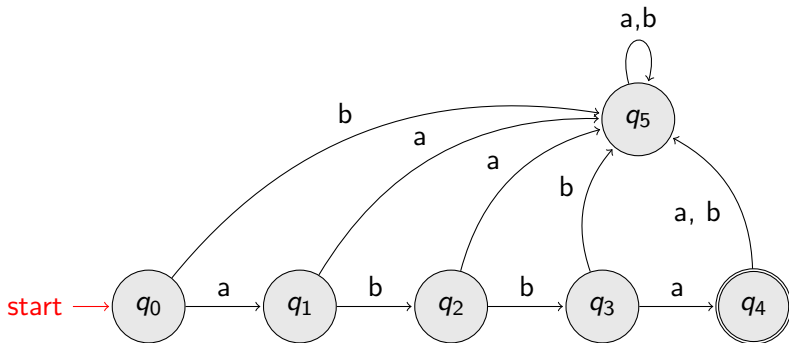
δ : Transition Function: $\delta : Q \times \Sigma \rightarrow Q$

q_0 : Initial State

F : Finite set of final states: $F \subseteq Q$

- Finite set of states, Q :

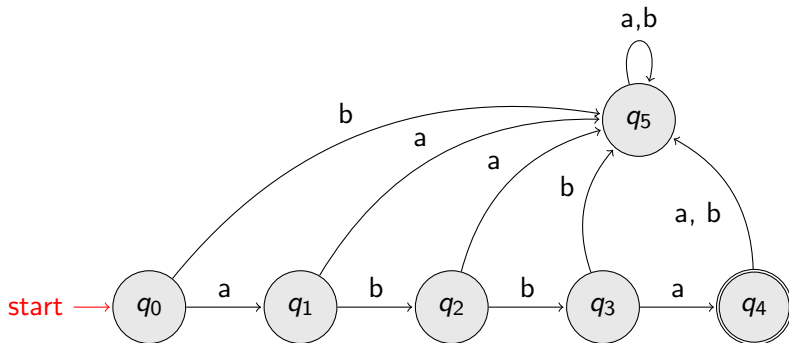
$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$



Formalities

- **Input Alphabet, Σ :**

$$\Sigma = \{a, b\}$$

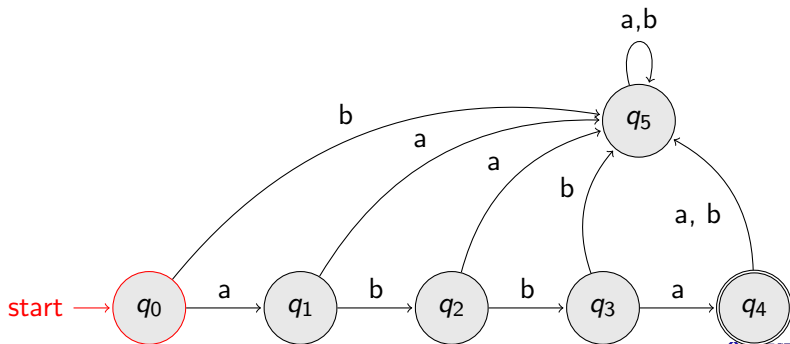


- **Initial State, $q_0 \subseteq Q$**
- **Finite Set of Final States, $F: F = \{q_4\} \subseteq Q$**

Formalities

- **Transition Function** , δ :

$$\delta : Q \times \Sigma \rightarrow Q$$

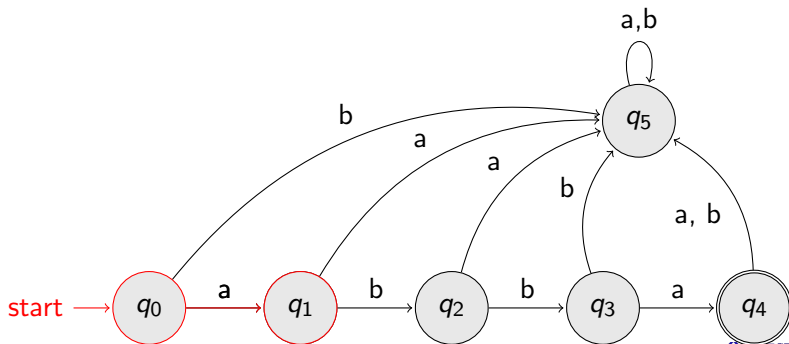


Formalities

- **Transition Function** , δ :

$$\delta : Q \times \Sigma \rightarrow Q$$

$$\delta(q_0, a) = q_1$$



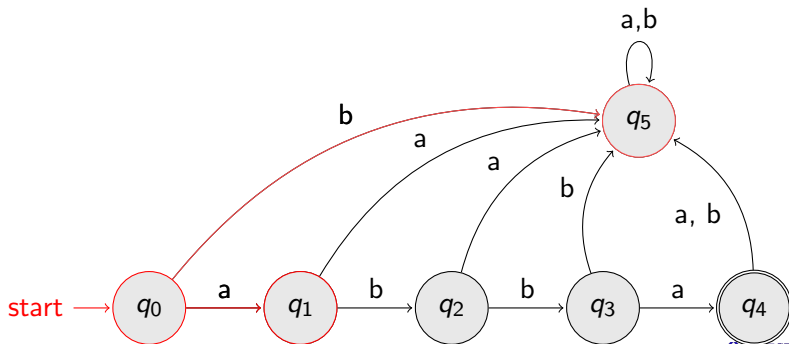
Formalities

- Transition Function , δ :

$$\delta : Q \times \Sigma \rightarrow Q$$

$$\delta(q_0, a) = q_1$$

$$\delta(q_0, b) = q_5$$



Formalities

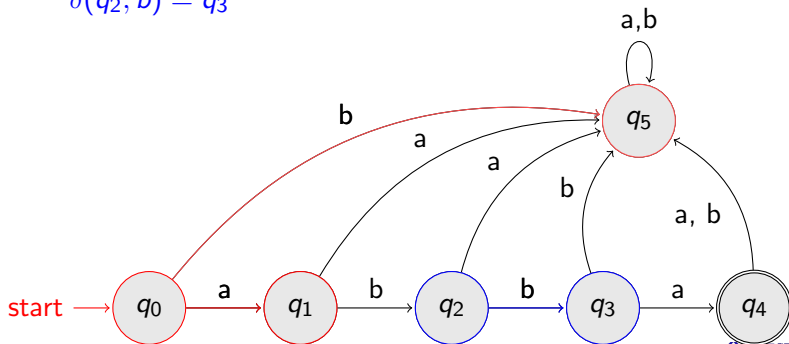
- Transition Function , δ :

$$\delta : Q \times \Sigma \rightarrow Q$$

$$\delta(q_0, a) = q_1$$

$$\delta(q_0, b) = q_5$$

$$\delta(q_2, b) = q_3$$

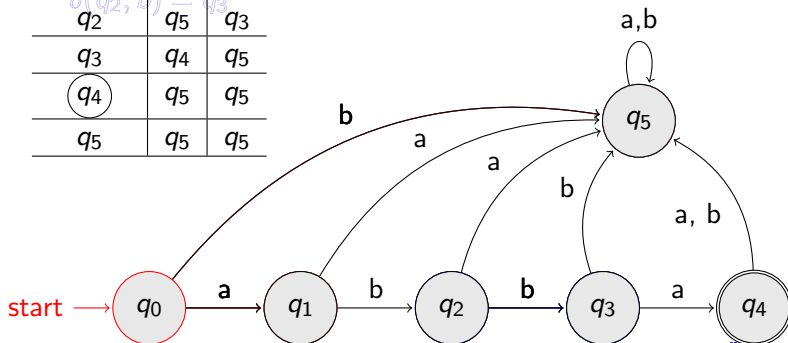


Formalities

- Transition Function , δ :

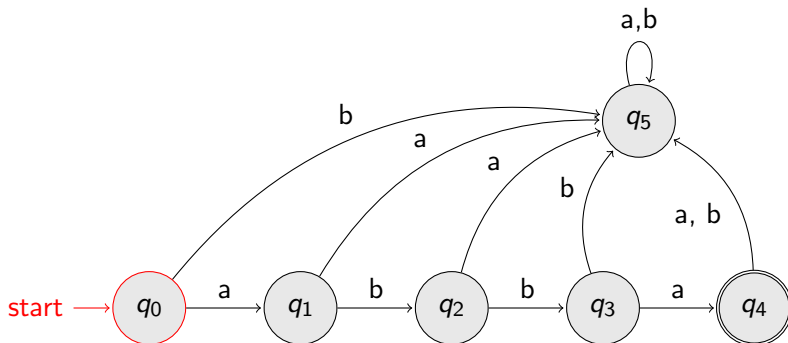
$$\delta : Q \times \Sigma \rightarrow Q$$

States	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5



- Extended Transition Function , δ^* :

$$\delta^* : Q \times \Sigma^* \rightarrow Q$$

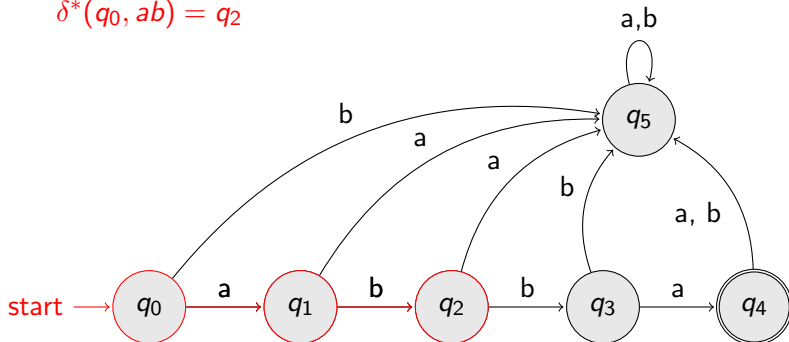


Formalities

- Extended Transition Function , δ^* :

$$\delta^* : Q \times \Sigma^* \rightarrow Q$$

$$\delta^*(q_0, ab) = q_2$$



- Extended Transition Function , δ^* :

$$\delta^* : Q \times \Sigma^* \rightarrow Q$$

$$\delta^*(q_0, ab) = q_2$$

$$\delta^*(q_0, abba) = q_4$$

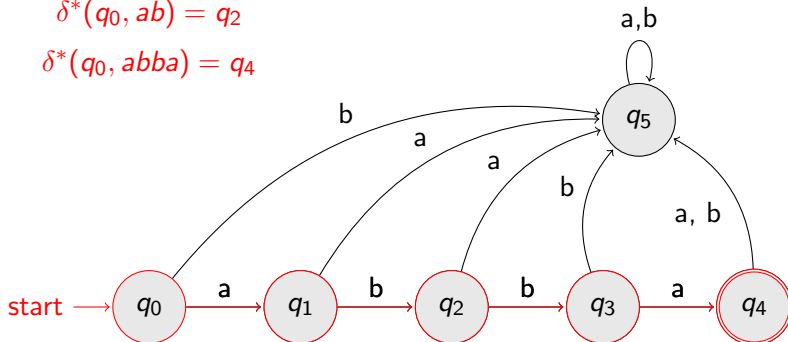


Table of Contents

- 1 Introduction to Strings
- 2 Languages
- 3 Finite Automata
- 4 Languages Accepted by DFAs

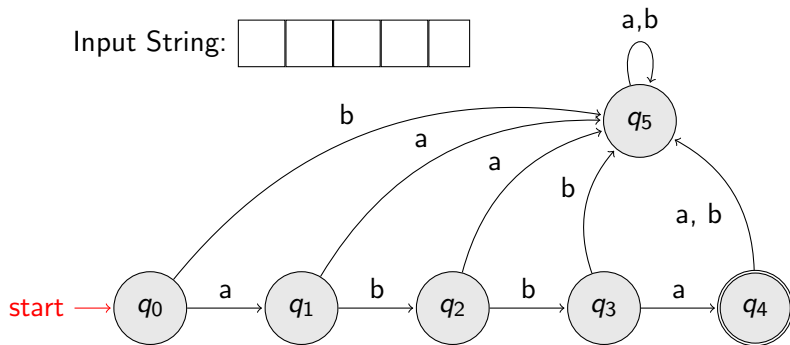
Languages Accepted by DFAs

- Consider a DFA, M

- The language, $L(M)$ contains all input strings accepted by M .
- $L(M) = \{ \text{strings that drive } M \text{ to Final State} \}$

- Example 1:

$$L(M) = \{abba\}$$

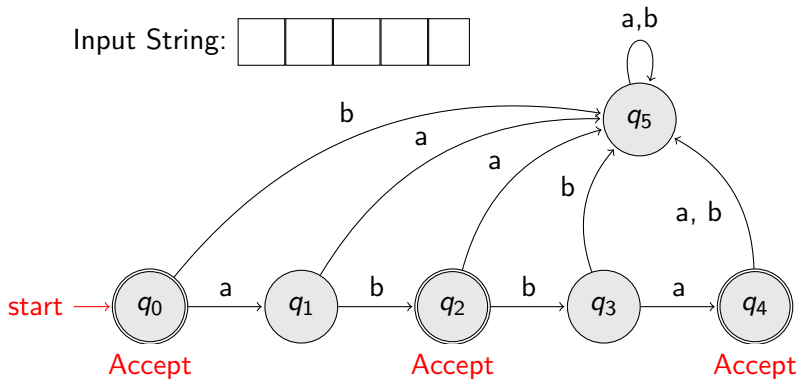


Languages Accepted by DFAs

- Example 2: $L(M) = \{\lambda, ab, abba\}$

Input String:

--	--	--	--	--



Languages Accepted by DFAs

- The language accepted by a DFA, $M = (Q, \Sigma, \delta, q_0, F)$ can be defined as follows:

- $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$

Here,

Q : Finite set of states

Σ : Input alphabet

δ : Transition function: $\delta^* : Q \times \Sigma^* \rightarrow Q$

q_0 : Initial State

F : Finite set of final states

Languages Accepted by DFAs

- **Observations:**

- Language **accepted** by a DFA, $M = (Q, \Sigma, \delta, q_0, F)$:
 $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$
- Language **rejected** by a DFA, $M = (Q, \Sigma, \delta, q_0, F)$:
 $\overline{L(M)} = \{w \in \Sigma^* : \delta^*(q_0, w) \notin F\}$

References

- Denial I. A. Cohen Introduction to Computer Theory, Second Edition, John Wiley & Sons, 1997.
- J. E. Hopcroft, R. Motwani, & J. D. Ullman Introduction to Automata Theory, Languages, and Computation, Third Edition, Pearson, 2008.